

Lecture 11 — From a Messy File to a Gradebook

Data cleaning, grading schemes, and the start of an object-oriented design

Amir Farbin

The problem

Today's problem

- A TA hands you a dump of everyone's scores. Grades are due.
- No software, no time, and the file is a mess
- A silly problem **on purpose**: everything real is in how we solve it
- Three levels today:
 - get the data usable
 - decide what a grade *means*
 - build something that survives being used again

→ open `Data-1401-Grades.csv`

Rule zero: look at the file

- `!ls -lh` — how big? Can I even open it?
- `!cat` — what is actually in there?
- What's wrong here:
 - first line is grading notes, not a header
 - words where numbers belong: *no solution, did not submit*
 - scores written 68-sent — the TA's note that it was emailed
 - blank cells everywhere

→ cells 2, 4

Rule zero: look at the file — cell 2

```
!ls -lh
```

```
total 736
```

```
-rw-----@ 1 afarbin  staff    8.5K Sep 16 11:38 Data-1401-Grades-Fixed.csv  
-rw-----@ 1 afarbin  staff    8.7K Sep 16 11:38 Data-1401-Grades.csv  
-rw-----@ 1 afarbin  staff   324K Sep 23 09:45 Lecture.11.ipynb  
-rw-----@ 1 afarbin  staff    37K Feb 25  2026 Lecture.11.pdf
```

Rule zero: look at the file — cell 4

```
!cat Data-1401-Grades.csv
```

```
"No attempt: -5,wrong:-1...-5","No attempt: -5, wrong: -2","No attempt: -10, wrong: ...  
Lab 1,Lab 2,Lab 3,Lab 4,Exam 1,Lab 5,,Lab 7,Exam 2  
,0,0,,0,0,0,0,0  
no solution,no solution,-did not submit the Lab,,did not submit the exam,,,,  
100,96,46,,68-sent,,,,  
Completed the lab,"ex4- attempted,wrong solution -2  
ex8 -wrong solution(sequence not correct) -2","ex1 - Nameerror get_size() is not def..  
ex2 - returns -1 for all conditions, does not test for all conditions.(-3)  
ex3 - does not construct board(-5)  
... (171 more lines)
```

Cleaning

Fix it from the shell, not by hand

- Never edit the data file by hand — you can't repeat it, and you can't explain it
- `!wc -l` to count, `!tail` to drop the bad first line, `>` to write a new file
- Keep the original. Write `Data-1401-Grades-Fixed.csv`
- Same tools they just used in the Lab 5 command-line session

→ cells 9, 11, 13, 15

Fix it from the shell, not by hand — cell 9

```
!wc -l Data-1401-Grades.csv
```

```
179 Data-1401-Grades.csv
```

Fix it from the shell, not by hand — cell 11

```
!tail -179 Data-1401-Grades.csv
```

```
Lab 1,Lab 2,Lab 3,Lab 4,Exam 1,Lab 5,,Lab 7,Exam 2
,0,0,,0,0,0,0,0
no solution,no solution,-did not submit the Lab,,did not submit the exam,,,
100,96,46,,68-sent,,,,
Completed the lab,"ex4- attempted,wrong solution -2
ex8 -wrong solution(sequence not correct) -2","ex1 - Nameerror get_size() is not def...
ex2 - returns -1 for all conditions, does not test for all conditions.(-3)
ex3 - does not construct board(-5)
ex4 - does not construct the board (-5)
... (170 more lines)
```

Fix it from the shell, not by hand — cell 13

```
!tail -179 Data-1401-Grades.csv > Data-1401-Grades-Fixed.csv
```

Fix it from the shell, not by hand — cell 15

```
!cat Data-1401-Grades-Fixed.csv
```

```
Lab 1,Lab 2,Lab 3,Lab 4,Exam 1,Lab 5,,Lab 7,Exam 2
,0,0,,0,0,0,0,0
no solution,no solution,-did not submit the Lab,,did not submit the exam,,,
100,96,46,,68-sent,,,,
Completed the lab,"ex4- attempted,wrong solution -2
ex8 -wrong solution(sequence not correct) -2","ex1 - Nameerror get_size() is not def...
ex2 - returns -1 for all conditions, does not test for all conditions.(-3)
ex3 - does not construct board(-5)
ex4 - does not construct the board (-5)
... (170 more lines)
```

pandas reads it properly

- `pd.read_csv` — now the header is a header
- **DataFrame** = a table; **Series** = one column
- A Series looks like a NumPy array but carries labels
- `df["Lab 1"]` — one column; the column names came from the file

→ cells 17, 18, 20

```
df = pd.read_csv("Data-1401-Grades-Fixed.csv")
```

pandas reads it properly — cell 18

```
df
```

	Lab 1 \
0	NaN
1	no solution
2	100
3	Completed the lab
4	100
5	Completed the lab
6	93
7	ex4 - attempted, wrong answer(-2)\nex5c - not ...
...	(194 more lines)

pandas reads it properly — cell 20

```
df["Lab 1"]
```

```
0          NaN
1      no solution
2          100
3      Completed the lab
4          100
5      Completed the lab
6          93
7      ex4 - attempted, wrong answer(-2)\nex5c - not ...
8          0
... (24 more lines)
```


Selecting rows

- `df[...]` is a very smart `__getitem__` — exactly Lecture 10's material
 - a string gives a column
 - a column of True/False gives rows
- Build a mask, apply it, get a new DataFrame back
- Assigning to a name that doesn't exist **creates a column**

→ cells 24, 25, 33

```
df_0=dfask]
```

Selecting rows — cell 25

df_0

	Lab 1	Lab 2	Lab 3	Lab 4	Exam 1	Lab 5	Unnamed: 6	Lab 7	Exam 2
0	NaN	0	0	NaN	0	0.0	0.0	0.0	0.0
2	100	96	46	NaN	68-sent	NaN	NaN	NaN	NaN
4	100	100	90	NaN	88-sent	NaN	NaN	NaN	NaN
6	93	100	24	NaN	96-sent	NaN	NaN	NaN	NaN
8	0	81	0	0	43-sent	NaN	NaN	NaN	NaN
10	95	100	105	NaN	96-sent	NaN	NaN	NaN	NaN
12	100	98	87	NaN	89-sent	NaN	NaN	NaN	NaN
14	95	98	50	NaN	52	NaN	NaN	NaN	NaN
... (8 more lines)									

Selecting rows — cell 33

df_0

	Lab 1	Lab 2	Lab 3	Lab 4	Exam 1	Lab 5	Unnamed: 6	Lab 7	Exam 2	\
0	NaN	0	0	NaN	0	0.0	0.0	0.0	0.0	
2	100	96	46	NaN	68-sent	NaN	NaN	NaN	NaN	
4	100	100	90	NaN	88-sent	NaN	NaN	NaN	NaN	
6	93	100	24	NaN	96-sent	NaN	NaN	NaN	NaN	
8	0	81	0	0	43-sent	NaN	NaN	NaN	NaN	
10	95	100	105	NaN	96-sent	NaN	NaN	NaN	NaN	
12	100	98	87	NaN	89-sent	NaN	NaN	NaN	NaN	
14	95	98	50	NaN	52	NaN	NaN	NaN	NaN	

... (26 more lines)

60–80% of the work is this

- Cleaning is most of the job, on every real dataset
- Fix the *format*, not the numbers
- Better: go back to whoever produced the file and fix it at the source
- Formats drift — look at the file every time, even when it's “the same” file

What the numbers say

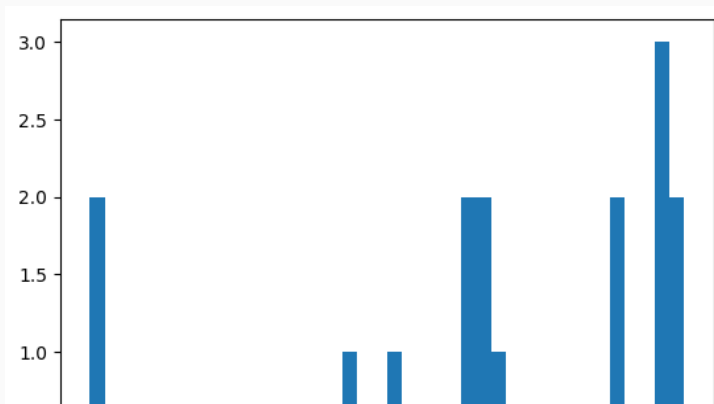
Histogram first

- A column of scores is a **random variable**; a histogram estimates its distribution
- Always look before you compute a statistic
- This one isn't normal — it's lumpy
 - mixed populations: students who did the work and students who didn't
 - the CLT doesn't save you when it's a mixture
- Mean and σ describe a distribution **only when it's normal**

→ cells 37, 38, 47

Histogram first — cell 37

```
%matplotlib inline  
import matplotlib.pyplot as plt  
  
plt.hist(df_0["Exam 1 Fixed"],bins=40)
```



Histogram first — cell 38

```
df_0["Exam 1 Fixed"]
```

```
0      0
```

```
2     68
```

```
4     88
```

```
6     96
```

```
8     43
```

```
10    96
```

```
12    89
```

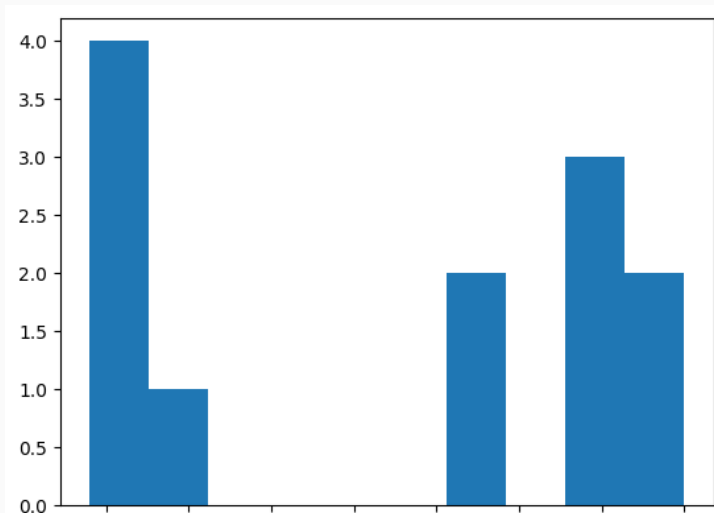
```
14    52
```

```
16   100
```

```
... (8 more lines)
```

Histogram first — cell 47

```
plt.hist(exam_1_df["Exam 1 Fixed"])
```



What is a grade for?

- A grade groups students who performed similarly — that's all it is
- Straight scale: 90/80/70 — assumes mean 80, σ 10. Says who told you that?
- Curve: map the class onto a normal distribution
- Normalized percentile: rank, then cut
- **The same score must get the same letter.** Pick one scheme and defend it

Counting in ranges

- `in_range(a,b)` returns a *function* — a closure, as in Lab 3
- Every way of counting scores in a range is **histogramming**
- Grade boundaries are just **bin edges**
- `plt.hist(..., bins=[0,50,64,80,90,101])` — the scheme *is* the bin list

→ cells 51, 56, 72

```
def in_range(a,b):  
    def func(x):  
        return x<b and x>=a  
    return func
```

Counting in ranges — cell 56

```
plt.hist(df_0["Exam 1 Fixed"],bins=[0,50,64,80,90,101])
```



Counting in ranges — cell 72

```
plt.hist(df_0["Exam 1 Fixed"],bins=list(grades.values()))
```



Summarize and curve

- `grade_summary` — how many in each letter, as a fraction
- `min_grade` — what floor do you set, and why?
- `compute_curve` — shift and scale to a target mean and spread
 - the same shift-and-scale as Lab 3's distribution exercise
- A is capped at the max; no one gets more than the best score

→ cells 74, 75, 76, 80, 81

Summarize and curve — cell 74

```
def grade_summary(df_0,grades,min_grade=0.):  
    df=df_0[df_0>min_grade]  
    N_total=sum(df>min_grade)  
    hist,bins=np.histogram(df,bins=list(grades.values()))  
  
    print("Mean:",np.mean(df))  
    print("Std:",np.std(df))  
  
    for grade_letter,bin_count in zip(grades.keys(),hist):  
        print(grade_letter+": "+str(bin_count),  
              float(bin_count/N_total) )
```

```
grade_summary(df_0["Exam 1 Fixed"],grades)
```

```
Mean: 77.64285714285714
```

```
Std: 18.38713503224136
```

```
F: 1 0.07142857142857142
```

```
D: 1 0.07142857142857142
```

```
C: 5 0.35714285714285715
```

```
B: 2 0.14285714285714285
```

```
A: 5 0.35714285714285715
```

```
grade_summary(df_0["Exam 1 Fixed"],grades,55)
```

```
Mean: 82.66666666666667
```

```
Std: 14.642024753731599
```

```
F: 0 0.0
```

```
D: 0 0.0
```

```
C: 5 0.4166666666666667
```

```
B: 2 0.16666666666666666
```

```
A: 5 0.4166666666666667
```

Summarize and curve — cell 80

```
def compute_curve(df_0, min_grade=None):

    if min_grade:
        pass
    else:
        min_grade=np.max(df_0)/2.

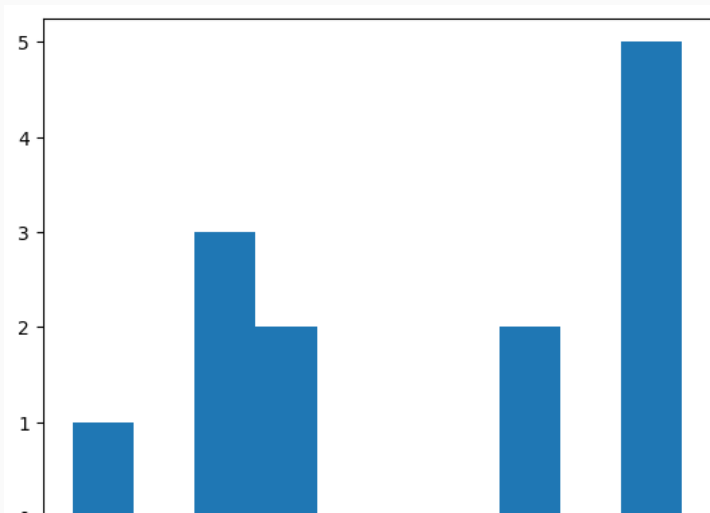
    print("Min grade:",min_grade)

    df=df_0[df_0>min_grade]
    N_total=sum(df_0>min_grade)

    print("N Total (post cut):",N_total)
    print("N Total (pre cut):",df_0.shape)
... (29 more lines)
```

Summarize and curve — cell 81

```
compute_curve(df_0["Exam 1 Fixed"],50.)
```



When it breaks: the `TypeError`

- Run the curve on Lab 1 and it **fails** — show the traceback
- Read it: comparing `str` and `float`
- `list(map(type, ...))` — the column is full of strings
- Why: comment rows made pandas read the whole column as text
- `fillna(0), astype` — now the numbers are numbers

→ cells 87, 89, 84, 85

When it breaks: the TypeError — cell 87

```
compute_curve(df_1["Lab 1"],50.)
```

```
Min grade: 50.0
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[58], line 1  
----> 1 compute_curve(df_1["Lab 1"],50.)  
  
Cell In[52], line 10, in compute_curve(df_0, min_grade)  
      6      min_grade=np.max(df_0)/2.  
... (37 more lines)
```

When it breaks: the TypeError — cell 89

```
list(map(type,df_1["Lab 1"]))
```

```
[int,  
 str,  
 str,  
 str,  
 str,  
 str,  
 str,  
 str,  
 str,  
 ... (7 more lines)
```


When it breaks: the `TypeError` — cell 84

```
df_1=df_0.fillna(0)
```

When it breaks: the TypeError — cell 85

df_1

	Lab 1	Lab 2	Lab 3	Lab 4	Exam 1	Lab 5	Unnamed: 6	Lab 7	Exam 2	\
0	0	0	0	0	0	0.0	0.0	0.0	0.0	
2	100	96	46	0	68-sent	0.0	0.0	0.0	0.0	
4	100	100	90	0	88-sent	0.0	0.0	0.0	0.0	
6	93	100	24	0	96-sent	0.0	0.0	0.0	0.0	
8	0	81	0	0	43-sent	0.0	0.0	0.0	0.0	
10	95	100	105	0	96-sent	0.0	0.0	0.0	0.0	
12	100	98	87	0	89-sent	0.0	0.0	0.0	0.0	
14	95	98	50	0	52	0.0	0.0	0.0	0.0	

... (26 more lines)

Assigning letters

- `assign_grade` walks the boundaries from the bottom up
- `map` applies it down the column — or a plain loop; both fine
- Weighted total across labs and exams → the class grade

→ cells 93, 96, 98, 99, 102

Assigning letters — cell 93

```
def assign_grade(numeric,grades):  
    previous_grade="F"  
    for grade_letter,grade_boundry in grades.items():  
        #print(numeric, grade_letter,grade_boundry)  
        if numeric < grade_boundry:  
            return previous_grade  
            break  
        else:  
            previous_grade=grade_letter  
    return previous_grade
```

Assigning letters — cell 96

```
def assign_grades(name,grades):  
    letter_grades=list()  
    for item in df_1[name].astype('int32'):  
        letter_grades.append(assign_grade(item, grades))  
    return letter_grades
```

Assigning letters — cell 98

```
df_1["Exam 1 Letter"]= assign_grades("Exam 1 Fixed",grades)
df_1
```

	Lab 1	Lab 2	Lab 3	Lab 4	Exam 1	Lab 5	Unnamed: 6	Lab 7	Exam 2	\
0	0	0	0	0	0	0.0	0.0	0.0	0.0	
2	100	96	46	0	68-sent	0.0	0.0	0.0	0.0	
4	100	100	90	0	88-sent	0.0	0.0	0.0	0.0	
6	93	100	24	0	96-sent	0.0	0.0	0.0	0.0	
8	0	81	0	0	43-sent	0.0	0.0	0.0	0.0	
10	95	100	105	0	96-sent	0.0	0.0	0.0	0.0	
12	100	98	87	0	89-sent	0.0	0.0	0.0	0.0	
14	95	98	50	0	52	0.0	0.0	0.0	0.0	

... (26 more lines)

Assigning letters — cell 99

```
df_1["Exam 1 Fixed"].map(lambda num_grade: assign_grade(num_grade,grades))
```

```
0      F
```

```
2      C
```

```
4      B
```

```
6      A
```

```
8      F
```

```
10     A
```

```
12     B
```

```
14     D
```

```
16    A+
```

```
... (8 more lines)
```

Assigning letters — cell 102

```
grade_max = 400
grade_def = [("A+", grade_max),
              ("A", .9*grade_max),
              ("B", .8*grade_max),
              ("C", .7*grade_max),
              ("D", .6*grade_max),
              ("F", 0) ]

grades=OrderedDict(grade_def[::-1])

grades_inverse=OrderedDict(zip(list(grades.values())[::-1],
                                list(grades.keys())[::-1]))
```


Building a system

Dissecting the task

- Every step is a **family of functions with the same inputs and outputs**:
 - statistics: grades $\rightarrow$ mean, σ , min
 - boundaries: stats $\rightarrow$ cut points
 - assignment: boundaries + grades $\rightarrow$ letters
 - validation: letters $\rightarrow$ fractions per grade
- Same signature, many implementations $\rightarrow$ that's a **class**

From a notebook to production

- Day 1: cut and paste in a notebook; the client wants an answer tomorrow
- Then: do it a hundred times → write functions
- Then: new data, not just this file → generalize
- Then: hand it to a TA who doesn't have your context → it needs an interface
- Finally: production, for users who aren't you
- Effort buys **reliability**, not features

Why not just write it fast?

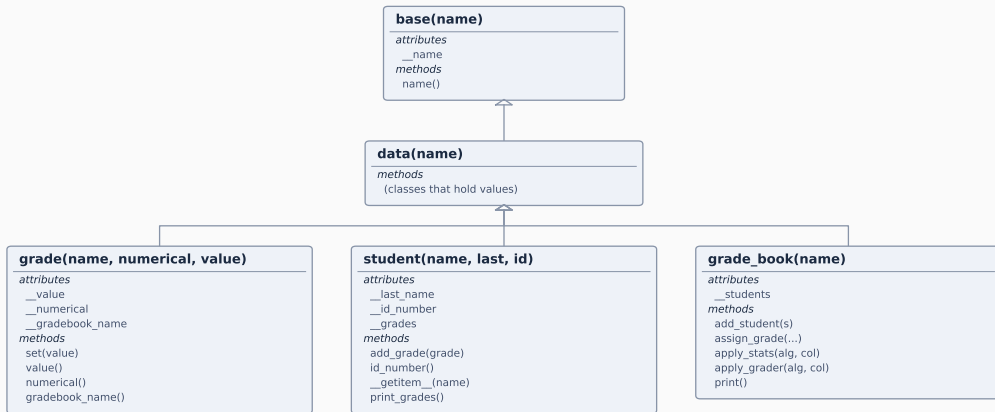
- Legacy code is the normal state: my experiment's ~7M lines of C++, 25 years old
- Interfaces outlive implementations — that's what you protect
- Data and algorithms stay in **separate classes**
 - the PDF format vs. the PDF reader
- Neither choice is obviously correct — avoid inventing generality you don't need

UML before code

- Boxes: class name, attributes, methods
- Arrows: inheritance
- Draw it first — cheaper to erase a box than to refactor code
- Tools generate the skeleton from the diagram
- Lab 6 asks for exactly this, before any blackjack code

The design, before the code

Data classes: they hold values



Every class carries a name, so an instance can be found and reported on.

The design

Base classes

- `base(name)` — everything has a name, so bookkeeping and lookup work
- `data` — things that hold values
- `alg` — things that do work
- Empty classes are fine as scaffolding: they mark where behavior will go (persistence, verbosity) without committing yet
- `__name` is private; the mangled `_base__name` is Lecture 10's name mangling

→ cell 108

One grade class, or two?

One class, a flag inside

grade(name, numerical=True, value=None)

attributes

`__value`
`__numerical`
`__gradebook_name`
`__letter_grades`

methods

`set(value)`
`numerical()`
`__str__()`

Two subclasses

grade(name, value=None)

attributes

`__value`
`__gradebook_name`

methods

`set(value)`
`__str__()`

letter_grade

attributes

`__letter_grades`

methods

`set(value)`

numerical_grade

methods

`set(value)`

Neither is obviously correct. The flag means one class to hold, and a branch in every method. The subclasses remove the branches, and add more classes than the problem needs yet.

Representing a grade and a student

- grade: value, numeric or letter, which gradebook column it came from
- One class with a flag, or two subclasses? **Neither is obviously right**
- student: identity fixed at construction; grades added later
- add_grade refuses to overwrite — a bad value should fail loudly, near where it got in

→ cells 117, 119

Representing a grade and a student — cell 117

```
class grade(data):  
    # These members are shared with all classes of the same type and doesn't b...  
    __letter_grades=["F-", "F", "F+", "D-", "D", "D+", "C-", "C", "C+", "B-", "B", "B+", "...  
  
    def __init__(self, name, numerical=True, value=None):  
        # These members belong to the instances of classes and not shared.  
        # Note all data members are private so we'll have to build accessors t...  
        self.__value=value  
        self.__numerical=numerical  
        self.__gradebook_name=str()  
  
        if value:  
            if isinstance(value, (int, float)):  
                self.__numerical=True  
  
... (32 more lines)
```

Representing a grade and a student — cell 119

```
class student(data):

    def __init__(self, first_name, last_name, id_number):
        self.__grades=dict()
        self.__id_number=id_number
        self.__first_name, self.__last_name = first_name, last_name
        data.__init__(self,first_name+" "+last_name+" Student Data")

    def add_grade(self,a_grade,overwrite=False):
        if overwrite or not a_grade.gradebook_name() in self.__grades:
            self.__grades[a_grade.gradebook_name()]=a_grade
        else:
            print (self.name()+" Error Adding Grade "+a_grade.name()+". Grade ...
            raise Exception

... (10 more lines)
```

Algorithms as classes

- `grade_calculator` — configuration in the constructor, work in `apply`
- `uncurved_letter_grade_percent`, `curved_letter_grade` — same interface, different policy
- `mean_std_calculator` — statistics
- `summary_calculator` — sums the weighted total
- Each holds only its *settings*; the data lives elsewhere

→ cells 123, 124, 127, 130, 135

Algorithms as classes — cell 123

```
class grade_calculator(alg):
    def __init__(self,name,stats):
        self.__stats=stats
        alg.__init__(self,name)

    def apply(self,a_grade):
        raise NotImplementedError

class uncurved_letter_grade_percent(grade_calculator):
    __grades_definition=[ (.97,"A+"),
                           (.93,"A"),
                           (.9,"A-"),
                           (.87,"B+"),
                           ... (36 more lines)
```

```
a_grader=uncurved_letter_grade_percent(grade_name="Exam 1 Test Grade")
out = a_grader.apply(a_student["Exam 1"])
print(type(out))
print(out)

<class '__main__.grade'>
Exam 1 Test Grade: A
```

Algorithms as classes — cell 127

```
class curved_letter_grade(grade_calculator):  
    __grades_definition=[ (.97,"A+"),  
                           (.93,"A"),  
                           (.9,"A-"),  
                           (.87,"B+"),  
                           (.83,"B"),  
                           (.8,"B-"),  
                           (.77,"C+"),  
                           (.73,"C"),  
                           (.7,"C-"),  
                           (.67,"C+"),  
                           (.63,"C"),  
                           (.6,"C-"),  
                           (.57,"F+"),  
  
    ... (38 more lines)
```



```
a_grader=curved_letter_grade(grade_name="Exam 1 Test Grade",mean=70.,std=10.)  
print( a_grader.apply(a_student["Exam 1"]))
```

Exam 1 Test Grade: B-

Algorithms as classes — cell 135

```
class summary_calculator(alg):
    def __init__(self,name):
        alg.__init__(self,name)

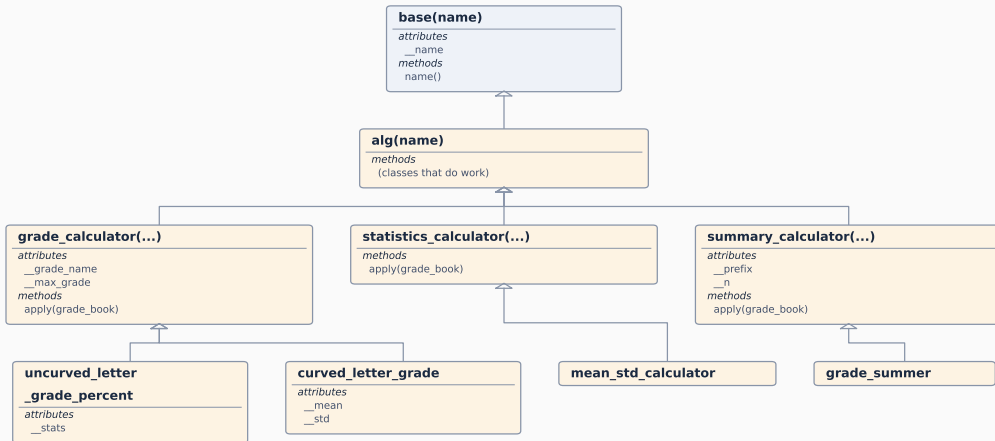
    def apply(self,a_student):
        raise NotImplementedError

class grade_summer(summary_calculator):
    def __init__(self,prefix,n):
        self.__prefix=prefix
        self.__n=n
        statistics_calculator.__init__(self,"Sum Grades")

    def apply(self,a_student):
... (7 more lines)
```

The calculators

Algorithm classes: same interface, different policy



Each subclass overrides apply(). Its own data is configuration, not the grades.

The gradebook

- `grade_book` holds students and their grades, and the results algorithms produce
- Load it from the DataFrame: rows $\rightarrow$ students, columns $\rightarrow$ grades
- `apply_stats`, `apply_grader`, `apply_summary` — run an algorithm over everyone
- Results get stored back in the book, so the next algorithm can use them

$\rightarrow$ cells 137, 140, 141, 142, 143

The gradebook — cell 137

```
class grade_book(data):

    def __init__(self, name):
        data.__init__(self, name+" Course Grade Book")
        self.__students=dict()

    def add_student(self, a_student):
        self.__students[a_student.id_number()]=a_student

    def assign_grade(self, key, a_grade):
        the_student=None
        try:
            the_student=self.__students[key]
        except:
            ... (28 more lines)
```

The gradebook — cell 140

```
a_grade_book=grade_book("Data 1401")

for student_i in range(df_0.shape[0]):
    a_student_0=student("Student",str(student_i),student_i)

    for k in df_0.keys():
        a_student_0.add_grade(grade(k,value=df_0[k].tolist()[student_i]))

a_grade_book.add_student(a_student_0)
```

```
a_grade_book.apply_stats(mean_std_calculator(),"Exam 1 Fixed")  
(67.9375, 30.906044615091073)
```

```
a_grade_book.apply_grader(curved_letter_grade("Exam 1 Letter Grade",68.,31.,ma...
```



```
a_grade_book.print()
```

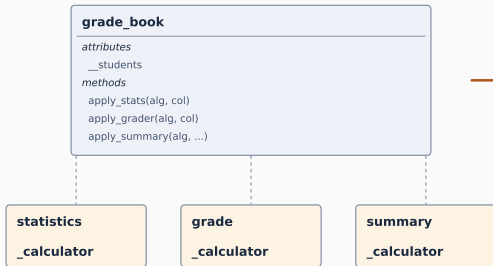
```
-----  
Student: 0  
-----  
Lab 1: nan  
Lab 2: 0  
Lab 3: 0  
Lab 4: nan  
Exam 1: 0  
Lab 5: 0.0  
... (215 more lines)
```

Where this is going

- It works. And the design already has a problem:
 - three kinds of calculator, three different `apply` methods
 - they differ only because we chose to make them differ
 - can't compose: curve, then normalize? Not without new code
- Lecture 12: collapse them into one `apply_calculator` interface
- That is the production-software lesson this chapter has been building to

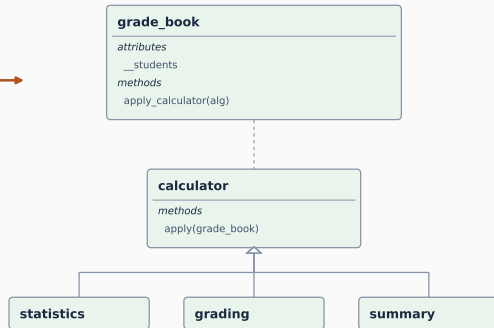
Lecture 12, in one picture

Lecture 11: three kinds, three methods



the caller has to know which kind it holds

Lecture 12: one interface



the grade book no longer cares